Introduction

Pre-pruning (also known as **early stopping**) restricts the growth of the tree '**during training**', preventing it ‘from reaching its maximum depth and potentially overfitting'.

So it stops growing the tree during training when splitting a node no longer improves the model meaningfully OR the node doesn’t meet any condition we have set manually. So to stop the growth of a tree we can set manual conditions **FOR EACH NODE** like :

- **max_depth = 5**: If the current level | depth of the tree is > 5, stop splitting.
- **min_samples_split = 20**: If node has < 20 samples, stop splitting.
- **min_impurity_decrease = 0.01**: If splitting reduces impurity (Gini/entropy) by < 0.01, stop.
- **min_samples_leaf = 10**: If any new leaf would have < 10 samples, cancel split.
- Splitting the node further won’t bring any significant improvement to the model.

If any of the conditions (i.e. threshold values of the corresponding hyperparameters. Also we can set more conditions) don't match for a node, that node becomes leaf node instant as it can’t anymore bring meaningful improvement to the model and we stop that node this way from growing more UNNESSERILY. That’s how “**Early Stopping”** happens for each node in Pre-Pruning process. Also that’s how overfitting is reduced/stopped through Pre-Prunning process.

Pre-Pruning Process **at Every Node**

- Can I split this node?
    - Does this node match **all** the Pre-pruning conditions (max_depth, max_features etc) we set for each node?
        - If it does, then calculate the best split using gini/entropy. Now does this split actually bring any meaningful improvement to the accuracy of the model?
            - If it does, then split on this node.
            - If it doesn't, the node becomes Leaf Node instant.
        - If it doesn't, the node becomes Leaf Node instant.

Repeat this step for each node until all branches end in leaf nodes (no node can/needs to split further).

All these are happening while constructing the tree.

Hyperparameter tuning using GridSearchCV

We do pre-pruning on a single tree during constructing it using a training dataset. But how do we know which max_depth, how many max_features to choose that will return the highest accuracy on testing dataset? Tuning|changing the values each time manually. But we can do it using GridSearchCV, e.g. :

```
parameters = { 'max_depth'         : [5, 7, 9],
               'max_features'      : [5, 9],
               'min_samples_split' : [6, 8, 9] }
```

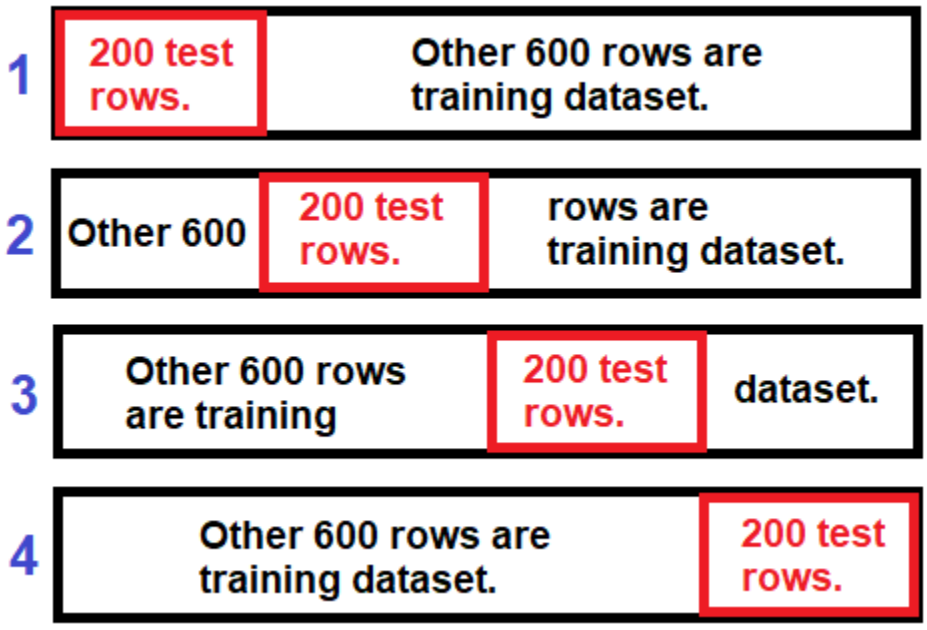
We can create 3 * 2 * 3 = 18 trees with different (max_depth, max_features, min_samples_split) combinations :

```
tree_1 = DecisionTree(max_depth = 5, max_features = 5, min_samples_split = 6)
tree_2 = DecisionTree(max_depth = 5, max_features = 5, min_samples_split = 8)
tree_3 = DecisionTree(max_depth = 5, max_features = 5, min_samples_split = 9)
```

tree_4 = DecisionTree(max_depth = 7, max_features = 5, min_samples_split = 6)
tree_5 = DecisionTree(max_depth = 7, max_features = 5, min_samples_split = 8)
tree_6 = DecisionTree(max_depth = 7, max_features = 5, min_samples_split = 9)
……
tree_18 = DecisionTree(max_depth= 9, max_features= 9, min_samples_split= 9)

As you can see there are 18 different trees with different **Pre-Pruning rules|conditions (max_depth, max_features, min_samples_split).** So, inside each tree, the Pre-Pruning process happens since each tree has Pre-Pruning rules|conditions. GridSearchCV creates all those 18 trees and at the end it returns the best values of (max_depth, max_features, min_samples_split) for which that tree returns the HIGHEST accuracy on testing data.

Usually we pass each Machine Learning model the training dataset to train and we predict/check_accuracy_of_that_model using testing dataset. GridSearchCV, a Hyperparameter Tuning Method, handles all of it automatically, so **we pass the main dataset(X, y) to it**. We use this method to find the best parameters from a given set of hyperparameters values. GridSearchCV has a parameter **cv(cross validation).** Assume the original data has 800 rows and cv = 4. So validation|testing rows = 8000 / 4 = 200 :



Here Cross Validation = 4 denoting "**Cross-checking Validation 4 times with 200 rows**". Each block is also called "**fold**".

Again, cv = 4 here also denoting "**4-fold CV: Dataset divided into 4 equal parts (200 test + 600 train per fold)**". Deepseek : (here 4 folds = 4 unique train/test splits)

Fold 1:
- Test = Partition 1 (200 rows)
- Train = Partitions 2+3+4 (600 rows)

Fold 2:
- Test = Partition 2 (200 rows)
- Train = Partitions 1+3+4 (600 rows)

Fold 3:
- Test = Partition 3 (200 rows)
- Train = Partitions 1+2+4 (600 rows)

Fold 4:
- Test = Partition 4 (200 rows)
- Train = Partitions 1+2+3 (600 rows)

So for each tree inside GridSearchCV, we're training the Tree Model with 600 rows and testing with 200 rows 4 times the way the dataset arranged in the above picture. And after doing Cross Validation 4 times, we return the mean accuracy to each tree model. That's how GridSearchCV trains and tests from the original|main dataset from its own. And after each tree model is trained and tested, GridSearchCV finds the best model using the accuracy score each tree model returned. By default GridSearchCV quantifies accuracy_score in each cross-validation which we can modify by tuning the parameter `scoring`.
Important parameters of GridSearchCV:

estimator : The model(linearRegression, DecisionTree) / method(Bagging/Boosting).

param_grid : The "parameters" variable in page 2.

scoring : The metrics(can be single or a list) GridSearchCV use in Cross-Validation for each model and also return them for each model.

refit : If "scoring" is a list i.e. multiple metrics, then comparing which metric(accuracy/rmse/mae….) we want to find the best model.

n_jobs : -1 if we want GridSearchCV to run itself using all the cores of our processor instead of a single core.

There's another method, RandomizedSearchCV which chooses some trees RANDOMLY from all the 9 trees and does calculations on those randomly chosen trees.